EE-568 - Applied Project 3: Imitation Learning Imitatio Group

Cyril Goffin
EPFL
SCIPER: 373937
cyril.goffin@epfl.ch

Emilien Coudurier
EPFL
SCIPER: 345431
emilien.coudurier@epfl.ch

Moritz Tschudin
EPFL
SCIPER: 327569
moritz.tschudin@epfl.ch

Abstract

This project compares imitation learning methods in low-demonstration regimes on CartPole-v1 and Acrobot-v1. A PPO expert is used to generate subsampled datasets with $K \in \{1, 3, 7, 10, 15\}$ trajectories. The main methods evaluated are IQ-Learn, CSIL, and CSIL+SOAR, with Behavior Cloning included as a supervised baseline and as the CSIL initialization. Results show that performance improves with K, but robustness differs strongly when demonstrations are scarce. IQ-Learn is particularly robust on CartPole-v1, while CSIL+SOAR performs best on Acrobot-v1 at $K = 1$, suggesting that optimistic critic ensembles can improve performance when only weak demonstrations are available.

1 Introduction

Imitation Learning (IL) aims to train agents from expert demonstrations instead of relying directly on manually specified reward functions. This is particularly useful when rewards are difficult to design, sparse, or misaligned with the desired behaviour, while examples from a competent expert are easier to interpret. In contrast to standard reinforcement learning, where the agent must discover good behaviour through trial and error, IL uses expert trajectories as supervision to guide policy learning.

In this project[1], the main comparison focuses on three dynamics-aware imitation learning methods: IQ-Learn, CSIL, and CSIL+SOAR. IQ-Learn learns an implicit reward through a soft Q-function, while CSIL derives a coherent shaped reward from an estimated expert policy and then fine-tunes it through a SAC-style procedure. Since CSIL requires an initial estimate of the expert policy, Behavior Cloning (BC) is also implemented and reported as a supervised baseline, but it is mainly used here as the initialization step on which CSIL and CSIL+SOAR build. The SOAR extension replaces the pessimistic twin-critic structure of CSIL with an optimistic critic ensemble, encouraging exploration in state-action regions where critic uncertainty is high.

The focus of this study is the sparse-demonstration setting, where only a very small number of expert trajectories is available. This regime is important because expert demonstrations can be costly to collect in practice, and because it highlights differences between algorithms that may be hidden when abundant expert data is available. To make the setting even more challenging, expert trajectories are temporally subsampled before training. Experiments are conducted on two Gymnasium control environments, CartPole-v1 and Acrobot-v1, using datasets with $K \in \{1, 3, 7, 10, 15\}$ expert trajectories.

[1] Our implementation is available at https://github.com/Cyril955/rl_course.

Preprint.

2 Related Work

In the past years, the scope of Imitation Learning (IL) spread and evolved with new, innovative strategies. Emerging as a way to improve the classic Supervised Learning (SL) methods, remarkable approaches including Inverse soft-Q Learning (IQ-Learn) and Coherent Soft Imitation Learning (CSIL) have shown appealing performance in various contexts.

Presented in [1], IQ-learn is an approach in which the agent is able to learn directly from the expert trajectories through an estimate of the Q-value obtained from the Bellman equation. This method bypasses the need for an explicit reward function and an expert policy generation, typical in Behavior Cloning (BC). With a rich enough trajectory dataset, IQ-learn has been shown to outperform numerous classical IL algorithms, offering significant flexibility.

On the other side, CSIL ([6]) leverages explicit reward identification and expert policy estimation to learn the optimal policy. This includes a BC step, constructing a virtual expert policy from the expert observation dataset, a coherent reward shaping from it, and a Soft Actor-Critic (SAC) sequence in order to benefit from the environment dynamics and explore in the expert policy direction. It is important to note that this exploration is exclusively due to the entropy term of SAC, and with a typical pessimistic twin-critic architecture, that the agent is most likely to stick to the BC-generated policy in environments where the optimal policy is far in the policy space. Similarly, in scenarios where attaining the global optimum demands discarding immediate reward, and therefore getting too far from the BC-generated policy, CSIL may fail. In other words, unless the expert dataset is rich enough for the BC policy to be close to optimal, CSIL lacks capabilities in terms of pure exploration.

For this reason, we exploit in this project what has been presented in [5], namely Soft Optimistic Actor cRitic (SOAR). This approach uses an ensemble of critics to then derive their mean and standard deviation for each action-state pair, and artificially increase expected reward in situations where uncertainty is high. Compared to a pessimistic twin-critic approach, SOAR pushes the agent to explore uncertain action-state pairs, allowing it to find more optimal policies. In the case of CSIL, this allows it to search for better performances, while keeping the BC policy direction as reference.

3 Approach

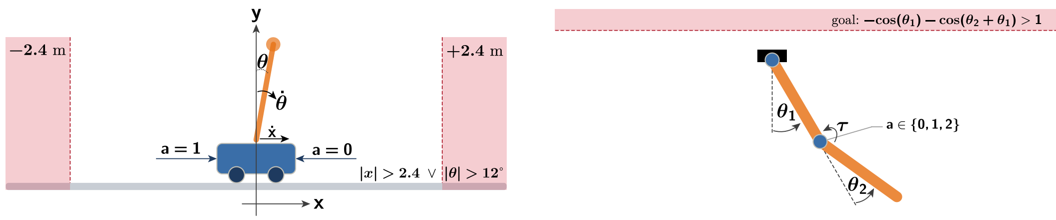
The objective of this study was to compare several imitation learning algorithms under sparse-demonstration conditions, where only a very small number of expert trajectories is provided. This setting was considered more realistic, since expert demonstrations are often expensive to collect in practical applications, and imitation learning agents must therefore be able to learn from incomplete and weakly representative datasets. To further increase the difficulty of the task and reduce temporal redundancy within individual demonstrations, the expert trajectories were sub-sampled before training, following the strategy used in [1]. The analysis was deliberately restricted to low-data regimes, with $K \in \{1, 3, 7, 10, 15\}$ trajectories, so that the convergence behaviour, robustness, and policy recovery capabilities of each algorithm could be assessed when expert supervision is extremely limited.

3.1 Environments

Using already implemented environments like we do with Gymnasium [4] environments offers convenient aspects: they are easy to implement, have extensive documentation and for classical environments, they are the base of many existing works to which we can compare. For this reason, we are using two typical Gymnasium environments in RL

- **Cartpole**: An inverted pendulum sitting on top of a translating cart as shown in Figure 1a. The action space is discrete where $a \in \{0, 1\}$, acting only on the cart movement. Its state is $s = \begin{bmatrix} x & \dot{x} & \theta & \dot{\theta} \end{bmatrix}^T \in \mathbb{R}^4$, taking into account the cart position and velocity, as well as the pole angle with the y-axis and its associated angular speed. The reward r_t at each timestep is simply $+1$, meaning a complete perfect episode obtains 500 as total reward since it terminates after 500 steps. If the cart goes out of $[-2.4, 2.4]$ or if the pole angle exceeds $12°$ in any direction, the episode terminates. The goal of the agent is therefore to balance the pole the longest.

- **Acrobot**: A double pendulum starting at its lowest position (see Figure 1b), where only the second joint is actuated. The action is also discrete, namely $a \in \{0, 1, 2\}$, translating to clockwise, counter-clockwise or no action. Its state, or equivalently the observational space is $s = \begin{bmatrix} cos(\theta_1) & sin(\theta_1) & cos(\theta_2) & sin(\theta_2) & \dot{\theta}_1 & \dot{\theta}_2 \end{bmatrix}^T \in \mathbb{R}^6$. The reward r_t at each timestep is -1, and terminates if the end effector exceeds the target height. This means that unlike the cartpole environment, the longer episode the worse the total reward becomes. Since the maximum number of steps per episode is 500, the worst reward possible is -500.



(a) CartPole-v1

(b) Acrobot-v1

Figure 1: Schematics for the CartPole-v1 and Acrobot-v1 environments provided by Gymnasium [4].

3.2 Expert

A Proximal Policy Optimization (PPO) [3] expert was trained to provide near-optimal demonstrations for the IL agents. PPO was chosen for its stability, fast convergence on simple control tasks, and well-documented implementation. Compared to REINFORCE, it benefits from lower-variance updates through generalized advantage estimation, while the additional sample efficiency of methods such as SAC was not necessary for these environments.

From the trained PPO expert, several expert datasets were extracted with different numbers of trajectories, following [1]: $K = 1, 3, 7, 10, 15$. Each trajectory consists of state-action pairs collected over one episode. To make the IL task more challenging and reduce temporal correlation within demonstrations, each trajectory was sub-sampled with frequency $f = 20$ for both environments, as illustrated in Figure 2.

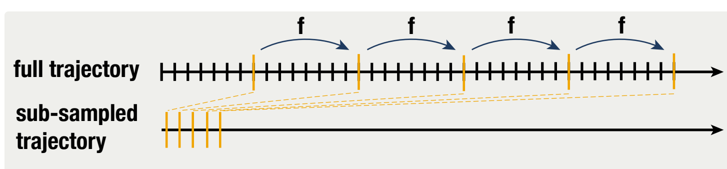


Figure 2: Temporal subsampling of expert demonstrations ($f = 8$ is shown for illustration).

This sub-sampling strategy does not produce equally rich datasets for both environments. CartPole expert episodes last 500 steps, whereas Acrobot expert episodes are typically shorter than 100 steps due to the termination condition. Consequently, the Acrobot IL agents receive roughly $5\times$ fewer state-action pairs, making imitation proportionally harder. This imbalance was kept intentionally: although fewer state-action pairs are available for Acrobot after sub-sampling, expert rollouts are shorter because the task is solved earlier under the termination condition.

The PPO expert is implemented using Stable-Baselines3[2], trained for 10^5 steps on CartPole-v1 and 3×10^5 steps on Acrobot-v1, with standard hyperparameters: learning rate 3×10^{-4}, rollout length 2048, mini-batch size 64, 10 epochs per rollout, GAE $\lambda = 0.95$, and clipping range 0.2. Once trained, the expert is rolled out *deterministically* to collect the pool of 15 trajectories, which are stored as $(s, a, r, s', \text{done})$ tuples; each IL algorithm then subsamples this pool down to its required K trajectories at frequency $f = 20$.

[2] https://github.com/DLR-RM/stable-baselines3

3.3 Imitation Learning Algorithms

3.3.1 BC

Behavior Cloning (BC) is not proper reinforcement learning but rather supervised learning directly applied to expert demonstrations. Although it is the simplest IL baseline, it is the foundation of several more advanced algorithms evaluated here, and assessing it explicitly lets us quantify the gains brought by each subsequent improvement (see Sections 3.3.3 and 3.3.4). BC maps states to actions by maximising the log-likelihood of expert actions:

$$\max_{\phi} \ \mathbb{E}_{(s,a)\sim\rho_E}\big[\log \pi_\phi(a \mid s)\big]. \tag{1}$$

It requires no environment interaction and is fast to train, but small prediction errors move the agent into unseen states where further errors compound, causing rapid policy degradation. This compounding effect is the main reason BC fails in practice with limited data. Nonetheless, with a sufficiently rich expert dataset and a simple enough task, BC can match or even outperform more complex methods.

BC is implemented as a 2-layer MLP (64 hidden units, ReLU activations) trained by minimising the cross-entropy loss against the subsampled expert actions, using Adam with learning rate 3×10^{-4}, batch size 256, and 600 epochs on both environments. No environment interaction takes place during training.

3.3.2 IQ-Learn

Inverse soft-Q Learning (IQ-Learn) [1] learns to imitate without access to the true environment reward and without adversarial training. The core idea is to encode an implicit reward directly inside a single soft Q-function Q_θ by inverting the soft Bellman equation:

$$\hat{r}(s,a) = Q_\theta(s,a) - \gamma V_\theta(s'), \tag{2}$$

where the soft value function is $V_\theta(s) = \alpha \log \sum_a \exp\big(Q_\theta(s,a)/\alpha\big)$ and α is the temperature. No reward network or discriminator is needed: the reward is implicit in Q_θ. The policy is then extracted with a SAC-style [2] entropy-regularised actor update:

$$\max_{\phi} \ \mathbb{E}_{s,a\sim\pi_\phi}\big[Q_\theta(s,a) - \log \pi_\phi(a \mid s)\big]. \tag{3}$$

This makes IQ-Learn dynamics-aware and stable to train, at the cost of requiring off-policy environment interactions. Q_θ is trained by minimising an objective derived from an f-divergence between the expert and the agent's occupancy measures. With the χ^2 divergence (our choice), the loss reads:

$$\mathcal{L}_{\mathrm{IQ}}(Q_\theta) = -\underbrace{\mathbb{E}_{\rho_E}\big[\hat{r}(s,a)\big]}_{\text{expert term}} + \underbrace{\mathbb{E}_{\rho_E}\big[V_\theta(s) - \gamma V_\theta(s')\big]}_{\text{Bellman consistency}} + \underbrace{\frac{1}{4\alpha}\,\mathbb{E}_{\rho_E}\big[\hat{r}(s,a)^2\big]}_{\chi^2 \text{ regulariser}}, \tag{4}$$

where ρ_E denotes the expert state-action distribution. The first term maximises Q-values on expert transitions, the second enforces Bellman self-consistency using expert states, and the χ^2 regulariser penalises large implicit rewards quadratically in closed form.

The Wasserstein-1 (W1) variant replaces the χ^2 term by a gradient penalty (GP) $\lambda\,\mathbb{E}\big[(\|\nabla_{(s,a)}Q_\theta(\hat{s},\hat{a})\|_2 - 1)^2\big]$, enforcing a 1-Lipschitz constraint on Q_θ. While W1 can help in high-dimensional continuous settings, it requires an extra backward pass through Q_θ for every update and introduces a sensitive penalty coefficient λ_{gp}. We opted for χ^2 because it yields a closed-form quadratic term with a single fixed hyperparameter α, is cheaper to compute, and is the default recommended variant in the original paper for simple discrete-action environments such as ours.

We rely on the IQ-Learn repository [3] from [1], adapting it to accept our PPO expert datasets. Expert trajectories are fed to the `SoftQ` agent after subsampling at frequency $f = 20$, matching the procedure applied to all other IL algorithms. Training proceeds online: the agent collects transitions into a replay buffer $\mathcal{B}_\pi$ while a fixed expert buffer $\mathcal{B}_E$ holds the K subsampled trajectories, at each gradient step mini-batches are drawn from both and concatenated. We use a 2-layer MLP with 64 hidden units, discount $\gamma = 0.99$, learning rate 10^{-4}, batch size 32, and a near-deterministic temperature $\alpha = 0.001$, with target network weights hard-updated every 4 gradient steps.

[3] https://github.com/Div99/IQ-Learn

3.3.3 CSIL

Coherent Soft Imitation Learning (CSIL) was introduced recently in [6]. It builds on BC to recover a shaped reward without adversarial training, and uses a SAC architecture to learn the expert policy. It can be broken down into three phases:

- **Phase 1 (BC):** A policy π_{bc} is trained on expert demonstrations, serving as an estimate of the expert true policy. It is used to warm-start the coefficient of the learned policy, and to direct the exploration.
- **Phase 2 (Coherent reward):** By inverting the entropy-regularised policy update, a shaped reward is derived for which π_{bc} is optimal:

$$\tilde{r}(s,a) = \alpha \log \frac{\pi_{\mathrm{bc}}(a \mid s)}{p(a \mid s)}, \tag{5}$$

  where p is a uniform prior over discrete actions. $\tilde{r}$ is recomputed online from the frozen π_{bc} at every SAC update.
- **Phase 3 (SAC fine-tuning):** SAC is run with $\tilde{r}$ as the sole reward signal. The actual environment reward is discarded, considered inaccessible. The actor is initialized from π_{bc}, making the coherent reward immediately informative. A pessimistic twin Q-critic with automatic entropy tuning is used for stability:

$$\tilde{Q}(s,a) = \min(Q_1, Q_2), \tag{6}$$

CSIL is implemented from scratch in PyTorch. The BC phase uses the same MLP and training procedure as the standalone BC baseline (2 layers, 64 hidden units, Adam at 3×10^{-4}, batch size 256, 600 epochs). The SAC phase runs for 1 500 episodes with a replay buffer of size 2×10^5, batch size 256, learning rate 3×10^{-4}, discount $\gamma = 0.99$, Polyak coefficient $\tau = 0.005$, and entropy temperature $\alpha = 0.1$. The actor is warm-started from π_{bc}, and $\tilde{r}$ is recomputed at every gradient step from the frozen π_{bc}.

3.3.4 SOAR-Enhanced CSIL

Soft Optimistic Actor cRitic (SOAR) is an approach proposed in [5] using an ensemble L of critics and leveraging their statistics to push the agent to explore uncertain state-action pairs. It therefore drives exploration by adding a bonus expected reward during computation of the estimate $\tilde{Q}(s,a)$:

$$Q^{\mathrm{opt}}(s,a) = \bar{Q}(s,a) + \mathrm{clip}\big(\sigma_Q(s,a),\, 0,\, \sigma\big), \tag{7}$$

Where $\bar{Q} = \frac{1}{L}\sum_\ell Q_\ell$ and σ_Q is the standard deviation across the ensemble. High disagreement increases the value of uncertain regions, pushing the policy to explore them.

This so-called SOAR enhancement can be applied to any critic-based architecture, and in our case will be applied to enhance CSIL. The pessimistic twin critic architecture is therefore discarded and a set of 4 critics is used instead. We highlight that the clipping std σ (chosen to be $\sigma = 10$ in our case) as well as this ensemble size are hyperparameters that should be the subject of a grid search to find the best set. However, due to time constraints, only a non-exhaustive search was led. This means the chosen values are most likely not optimal. We nevertheless consider them reasonable based on the reward's order of magnitude (the saturated bonus lies between 5% and 10% of the maximum absolute total reward) and since an ensemble of 4 critics was typically used in [5] for simple environments such as ours. Note that for too big ensembles, the consensus benefits disappear, rendering SOAR inefficient.

CSIL+SOAR is implemented as a direct extension of our CSIL codebase, with the twin-critic replaced by an ensemble of $L = 4$ independent Q-networks (2-layer MLP, 64 hidden units each). The optimistic value estimate Q^{opt} is computed at every actor update from the ensemble mean and clipped standard deviation with $\sigma = 10$. All other hyperparameters are identical to CSIL: 1 500 SAC episodes, replay buffer of size 2×10^5, batch size 256, learning rate 3×10^{-4}, $\gamma = 0.99$, $\tau = 0.005$, $\alpha = 0.1$, and the actor warm-started from the same pre-trained π_{bc}.

4 Results

We present results in three parts: training curves showing convergence across seeds and trajectory counts (Section 4.1), final evaluation returns as a function of K (Section 4.2), and a state-visitation analysis providing a geometric view of how algorithms explore the state space at $K = 1$ (Section 4.3).

4.1 Training

Figure 3 shows the episodic return over training for IQ-Learn (χ^2), CSIL, and CSIL+SOAR on both environments, for each $K \in \{1, 3, 7, 10, 15\}$. Each curve is averaged over 5 training seeds, shaded bands indicate ± 1 standard error of the mean. IQ-Learn is trained for a fixed number of gradient steps (100k for CartPole-v1, 200k for Acrobot-v1), while CSIL and CSIL+SOAR are trained for 1 500 SAC episodes on both environments, the horizontal axis is therefore normalised to $[0, 1]$. BC, Expert (PPO), and Random policies are shown as dashed horizontal baselines.

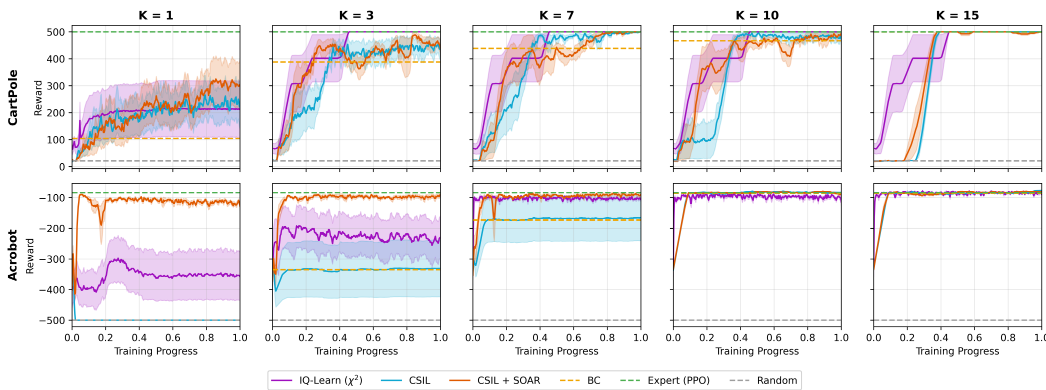


Figure 3: Training curves for IQ-Learn (χ^2), CSIL, and CSIL+SOAR on CartPole-v1 and Acrobot-v1, for each expert dataset size $K \in \{1, 3, 7, 10, 15\}$. Shaded bands show ± 1 standard error of the mean over 5 training seeds. BC, Expert (PPO), and Random baselines are shown as dashed horizontal lines.

4.2 Evaluation

Figure 4 reports the mean episode return at the end of training as a function of K, for all four algorithms on both environments. Each point aggregates 5 training seeds $\times$ 5 evaluation seeds = 25 independent runs, with each run averaged over 20 evaluation episodes, shaded bands show ± 1 standard error of the mean over these 25 runs.

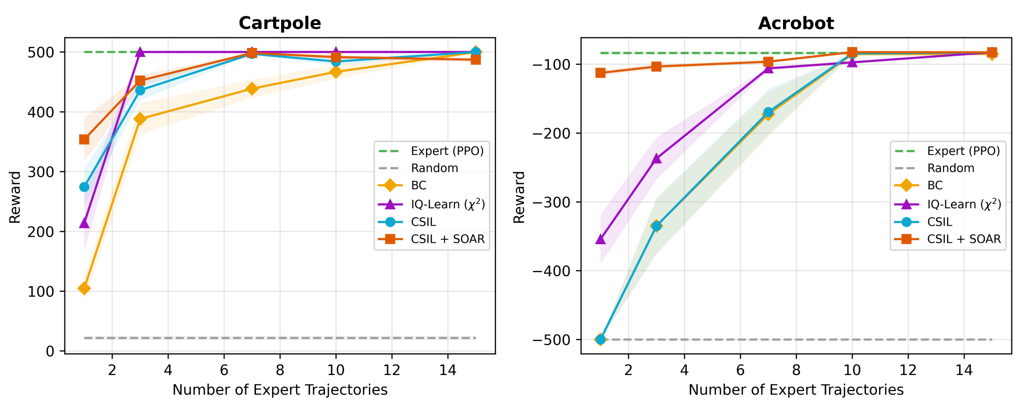


Figure 4: Evaluation performance of BC, IQ-Learn (χ^2), CSIL, and CSIL+SOAR as a function of the number of expert trajectories K, for CartPole-v1 and Acrobot-v1. Shaded bands show ± 1 standard error of the mean over 25 runs (5 training seeds $\times$ 5 evaluation seeds, 20 episodes each). Expert (PPO) and Random baselines are shown as dashed horizontal lines.

4.3 State Visitation on Acrobot-v1

For each method (PPO expert, IQ-Learn, BC, CSIL, CSIL+SOAR) we load the policies of all 5 training seeds and perform 20 deterministic (greedy) evaluation episodes per seed, yielding 100 episodes per method and 500 episodes in total. Acrobot's six-dimensional state decomposes cleanly into two joint angles $(\theta_1, \theta_2) \in [-\pi, \pi]^2$, recovered from the observation via $\theta_i = \operatorname{atan2}(\sin\theta_i, \cos\theta_i)$. This makes it possible to visualise directly where each trained policy goes in configuration space, providing a geometric view of behaviour that the aggregate returns cannot reveal. This is the state visitation map presented in Figure 5. All visited states are projected onto (θ_1, θ_2) and binned on a 60×60 grid, rendered as a log-scaled 2D histogram on a shared color scale. The goal-region boundary $-\cos\theta_1 - \cos(\theta_1 + \theta_2) = 1$, corresponding to the swing-up termination condition, is overlaid as a red dashed contour. The PPO panel shows the actual expert policy used to generate the demonstrations, the other four panels show policies trained from $K = 1.7$

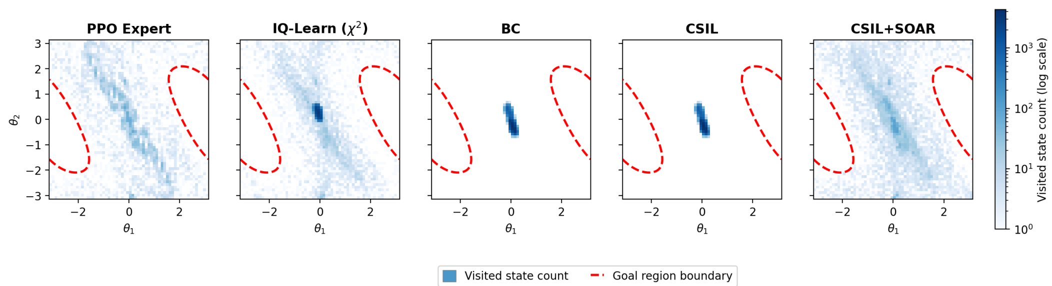


Figure 5: State visitation on Acrobot-v1 at $K = 1$. Each panel shows the log-scaled 2D histogram of (θ_1, θ_2) angles visited during greedy evaluation rollouts (5 training seeds $\times$ 20 episodes per seed = 100 episodes per method, 60×60 angular bins). Red dashed curves show goal-region boundary.

5 Discussion

5.1 Cartpole

The Expert (PPO) baseline sits at the maximum return of 500 across all K, confirming full convergence, while the Random baseline remains consistently low, providing a lower bound for comparison.

At $K = 1$, a clear performance ordering emerges: BC performs worst, as only a small number of subsampled transitions are available, which is insufficient to learn a reliable policy. IQ-Learn improves over BC but still struggles in this data-scarce regime, showing the largest variance among IL methods. CSIL outperforms IQ-Learn by building on the BC policy and augmenting it with environment interactions, and CSIL+SOAR achieves the highest return at $K = 1$ by further driving exploration through its optimistic ensemble critic, proving most robust under sparse demonstrations.

As K increases, all methods improve and variance narrows across seeds, reflecting more reliable learning with richer expert data. IQ-Learn reaches near-expert performance already at $K = 3$ and remains stable thereafter (see Figure 3). CSIL and CSIL+SOAR also approach expert performance but exhibit slight non-monotonic behaviour around $K = 10$, consistent with the higher variance visible in the training curves and attributable to the exploration-driven nature of the SOAR ensemble. BC improves steadily but does not fully close the gap to the expert even at $K = 15$, confirming the expected limitation of pure supervised imitation. At large K, all three IL methods converge near 500, suggesting that the choice of algorithm matters most in the low-data regime.

5.2 Acrobot

The Random baseline scores -500 with near-zero variance, never reaching the goal and exhausting the time limit every episode. The Expert (PPO) baseline reflects that even an optimal policy requires a variable number of steps to time the swing correctly.

At $K = 1$, BC performs poorly: the short average episode length of Acrobot combined with uniform subsampling leaves very few expert transitions per trajectory, making single-demonstration imitation particularly challenging. CSIL is superimposed on BC throughout, failing to improve over its warm-start: the pessimistic twin-critic architecture discourages the policy from diverging far from the BC initialisation, limiting exploration to what BC already covers. CSIL+SOAR, by contrast, achieves near-expert performance already at $K = 1$, standing out clearly from all other methods. Unlike CartPole, where the task could be largely solved by staying close to the expert, Acrobot requires broader state-space exploration to time the swing correctly, precisely what the optimistic ensemble critic enables. IQ-Learn sits between the two groups at low K, partially solving the task but with high variance.

As K increases, IQ-Learn improves consistently and reaches near-expert performance from $K \geq 7$. BC and CSIL also improve but remain superimposed, confirming that SAC fine-tuning in CSIL provides no benefit over plain BC in this environment. CSIL+SOAR remains the most sample-efficient method, maintaining near-expert performance across the full range of K.

5.3 State Visitation

The evaluation results (Figure 4) show that on Acrobot-v1 at $K = 1$ the three IL algorithms diverge most strongly from one another and from the BC baseline, motivating the state-visitation panels (Figure 5). BC and CSIL both cluster near the hanging-down start configuration ($\theta_1 \approx 0$, $\theta_2 \approx 0$), with no density reaching the goal region: both policies fail to initiate the swing and exhaust the 500-step time limit, producing the -500 return floor. The PPO expert exhibits a clear diagonal swing-up corridor that crosses the goal boundary. CSIL+SOAR is the only IL method at $K{=}1$ whose visited-state distribution spans a substantial portion of (θ_1, θ_2) space and reaches the goal region, closely matching the expert's footprint. This reflects the effect of the optimistic-ensemble bonus: by inflating Q-values at uncertain states, SOAR drives the policy beyond the BC-shaped reward attractor into regions the BC policy never visits. The complementary CartPole analysis in Appendix A shows that the same mechanism yields a much smaller return improvement on a stabilisation task, highlighting the task-dependence of SOAR's exploration benefit.

6 Conclusion

In this project, we compared different algorithms and assessed their respective performances on two classical RL environments. We highlighted their ability to learn a policy close to optimal when the number of expert demonstration is low and sparse. We conclude that simple Behavior Cloning suffers the most under these conditions, while IQ-Learn shows overall good performance across the set of trajectories. This is not the case for CSIL, which failed to improve over the BC baseline on Acrobot and did not demonstrate significant improvement on CartPole. Its SOAR enhanced version allowed CSIL to leverage an explorative behavior to outperform every algorithm on Acrobot, and presented a significative improvement on Cartpole, although its high exploration tendency decreased its stability. These analyses allowed us to identify trade-offs between different algorithms, regarding their returned reward and their computation time, emphasizing that computationally heavy algorithms do not always prevail.

Future work could extend this study in several directions. First, larger and intermediate values of K could be tested to better characterize both the low-data transition regime and the stability of each algorithm once near-expert performance is reached. Second, the comparison should be repeated on additional environments, including more complex or continuous-control tasks, to assess whether the observed trends generalize beyond CartPole-v1 and Acrobot-v1. Additionally, a more systematic hyperparameter search would strengthen the conclusions, since the present study relied on reasonable but non-exhaustively tuned configurations. This includes common parameters such as learning rates, batch sizes, entropy temperatures, and training horizons, as well as algorithm-specific choices such as the SOAR critic ensemble size, the optimistic bonus clipping value, or the IQ-Learn temperature and divergence variant. Finally, a quantitative study regarding the computational cost of each algorithm and their time/space complexity may be necessary to properly compare them.

AI Declaration

Generative AI tools (ChatGPT, Claude) were used during this project for three purposes: clarifying algorithmic concepts from the referenced papers when the original formulations were not straightforward to understand, assisting in debugging and understanding errors encountered during implementation, and improving the language of the report through grammar correction and text rephrasing. All scientific analyses, experimental designs, interpretations, and conclusions are solely the work of the authors.

References

[1] D. Garg, S. Chakraborty, C. Cundy, J. Song, and S. Ermon. Iq-learn: Inverse soft-q learning for imitation. *CoRR*, abs/2106.12142, 2021. URL `https://arxiv.org/abs/2106.12142`.

[2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018. URL `https://arxiv.org/abs/1801.01290`.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[4] M. Towers, A. Kwiatkowski, J. K. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024. URL `https://arxiv.org/abs/2407.17032`.

[5] S. Viel, L. Viano, and V. Cevher. IL-SOAR : Imitation learning with soft optimistic actor critic. In *Forty-second International Conference on Machine Learning*, 2025. URL `https://openreview.net/forum?id=NNr8DHb0L7`.

[6] J. Watson, S. Huang, and N. Heess. Coherent soft imitation learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 14540–14583. Curran Associates, Inc., 2023. URL `https://proceedings.neurips.cc/paper_files/paper/2023/file/2f0435cffef91068ced08d7c7d8e643e-Paper-Conference.pdf`.

A State Visitation on CartPole-v1

Following the same protocol as the Acrobot analysis (Section 4.3), we apply the state-visitation analysis to CartPole-v1 at $K = 1$. The CartPole state $s = (x, \dot{x}, \theta, \dot{\theta}) \in \mathbb{R}^4$ contains the cart position and the pole angle as direct components, so each visited state is projected onto (x, θ) without any nonlinear transformation. The plotting bounds are extended approximately 25% beyond the termination thresholds ($|x| = 2.4$ m, $|\theta| = 12°$) so that the safe-region rectangle, overlaid as a red dashed contour, is clearly visible inside each panel. For each method (PPO expert, IQ-Learn, BC, CSIL, CSIL + SOAR) the policies trained from all 5 seeds are rolled out for 20 deterministic evaluation episodes per seed, yielding 100 episodes per method and 500 episodes in total, and the visited states are binned on a 60×60 grid as a log-scaled 2D histogram (Figure 6). In contrast to Acrobot, the optimal policy on CartPole minimises the deviation from the unstable equilibrium $(x, \theta) = (0, 0)$, so a smaller and more tightly concentrated footprint is the geometric signature of better balancing.

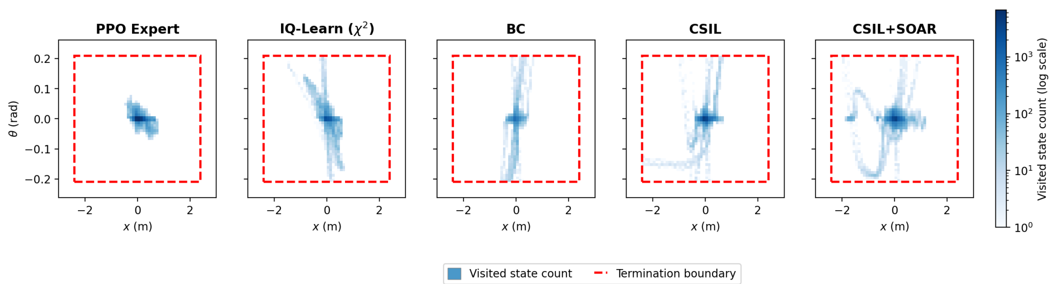


Figure 6: State visitation on CartPole-v1 at $K = 1$. Each panel shows the log-scaled 2D histogram of (x, θ) visited during greedy evaluation rollouts (5 training seeds $\times$ 20 episodes per seed = 100 episodes per method, 60×60 bins). Red dashed rectangle: termination boundary $|x| = 2.4$ m, $|\theta| = 12°$.

The PPO expert occupies a small, tight region around the origin with a slight diagonal stretch corresponding to the natural anti-correlation between pole angle and cart motion that an optimal balancing policy must maintain, the cart barely moves and the pole stays nearly upright. BC produces a very thin vertical band that extends along θ all the way to both termination edges, indicating that the policy fails to react to pole drift and lets it tip over, the narrow x extent reflects the absence of any cart-motion strategy. CSIL exhibits the same vertical band as BC, with a small horizontal extension near $\theta = 0$ that signals minor cart motion before failure, consistent with Section 5.1 finding that CSIL inherits its main behaviour from the BC warm-start and does not meaningfully diverge from it on this task. IQ-Learn shows a similar vertical band but with somewhat broader spread, consistent with its intermediate $K = 1$ return. CSIL+SOAR exhibits a substantially wider footprint than the other IL methods, with a dense central cluster around the origin and additional mass extending toward the edges of the safe region: the central concentration confirms that the policy does balance, while the wider extent reflects active cart motion driven by the optimistic ensemble bonus.

Comparing the two environments, SOAR's mechanism, inflating Q-values at uncertain state-action pairs and thereby widening the distribution of visited states, operates consistently across tasks. What differs is the payoff of this widening. On Acrobot the optimum lies far from the start, so widening the visited distribution is precisely what is needed to reach the goal region, and CSIL+SOAR achieves a near-expert $K = 1$ return where CSIL fails completely. On CartPole the optimum lies at the start, so the same widening yields only a marginal improvement over CSIL (Figure 4), SOAR remains the best $K = 1$ IL method, but the gap to CSIL is much smaller than on Acrobot. The state-visitation panels therefore confirm that SOAR's exploration mechanism is task-agnostic, while its return benefit is contingent on whether reaching states distant from the expert demonstration is necessary to solve the task.